

INFORME DE TESTING – PROYECTO FLASHU

Fecha: 21 de Noviembre, 2025

Equipo: FLASHU Dev Team

Tecnologías: React, Vitest, React Testing Library

1. Introducción y Propósito

El objetivo principal de este plan de pruebas es garantizar la estabilidad, seguridad y usabilidad de la aplicación web FLASHU. Nos hemos enfocado en validar los flujos críticos del negocio, asegurando que la interfaz responda correctamente ante las interacciones del usuario y que la lógica interna funcione sin errores.

Los puntos clave evaluados incluyen:

- Autenticación segura de usuarios.
 - Navegación fluida y protegida.
 - Gestión correcta del estado del carrito de compras.
 - Carga dinámica de productos desde la API simulada.
-

2. Herramientas y Configuración

Para la ejecución de las pruebas unitarias y de integración en el Frontend, se utilizó el siguiente stack tecnológico moderno:

- **Vitest:** Framework de testing de nueva generación, seleccionado por su velocidad y compatibilidad nativa con Vite.
- **React Testing Library (RTL):** Herramienta estándar para renderizar componentes y simular la interacción del usuario (clics, escritura) de manera accesible, tal como lo haría un usuario real.
- **JSdom:** Entorno de navegador simulado para ejecutar pruebas en Node.js sin necesidad de abrir un navegador real.

Ejecución de Pruebas

Las pruebas se ejecutan mediante los siguientes comandos en la terminal:

- `npm run test`: Ejecuta la suite completa en modo consola.
 - `npm run test:ui`: Despliega una interfaz gráfica interactiva para visualizar el estado de cada test en tiempo real.
-

3. Plan de Testing

Se seleccionaron 5 componentes/módulos críticos para someter a pruebas exhaustivas, asegurando la cobertura de las funcionalidades más importantes.

3.1. Login (Autenticación)

Objetivo: Verificar que el formulario de inicio de sesión se renderice correctamente y permita la interacción del usuario.

- **Archivo:** src/pages/Login/LoginPage.test.jsx
- **Pruebas Realizadas:**
 - Existencia de campos de correo y contraseña.
 - Capacidad de escribir en los inputs controlados.

Ejemplo de Código:

JavaScript

```
it('permite escribir en los inputs', () => {  
  renderLogin()  
  
  const emailInput = screen.getByPlaceholderText(/nombre@ejemplo.com/i)  
  
  fireEvent.change(emailInput, { target: { value: 'usuario@test.com' } })  
  
  expect(emailInput.value).toBe('usuario@test.com')  
})
```

3.2. Navbar (Navegación y Roles)

Objetivo: Asegurar que la barra de navegación muestre los enlaces correctos según el estado del usuario (logueado o invitado) y que el contador del carrito funcione.

- **Archivo:** src/components/common/PublicNavbar/PublicNavbar.test.jsx
- **Pruebas Realizadas:**
 - Visibilidad de enlaces públicos (Inicio, Catálogo).
 - Aparición del botón "Ingresar" solo para usuarios no autenticados.
 - Verificación del badge numérico del carrito.

3.3. Carrito de Compras (Lógica de Negocio)

Objetivo: Validar que los productos agregados se visualicen correctamente y que el sistema responda ante un carrito vacío o lleno.

- **Archivo:** src/pages/Public/CarritoPage.test.jsx

- **Pruebas Realizadas:**

- Mensaje de "Carrito vacío" cuando no hay productos.
- Renderizado correcto del nombre y precio formateado.
- Aparición del botón de pago solo cuando existen ítems.

3.4. Catálogo (Integración con API)

Objetivo: Verificar que la página de catálogo cargue los productos simulados y los muestre al usuario.

- **Archivo:** `src/pages/Public/CatalogoPage.test.jsx`

- **Pruebas Realizadas:**

- Mock de la llamada a la API `getProducts`.
- Verificación de que las tarjetas de producto aparecen en pantalla.
- Funcionamiento del botón "Agregar al carrito".

3.5. CartContext (Lógica de Estado Global)

Objetivo: Probar la lógica pura del carrito (agregar, eliminar, calcular total) de forma aislada, sin depender de la interfaz gráfica.

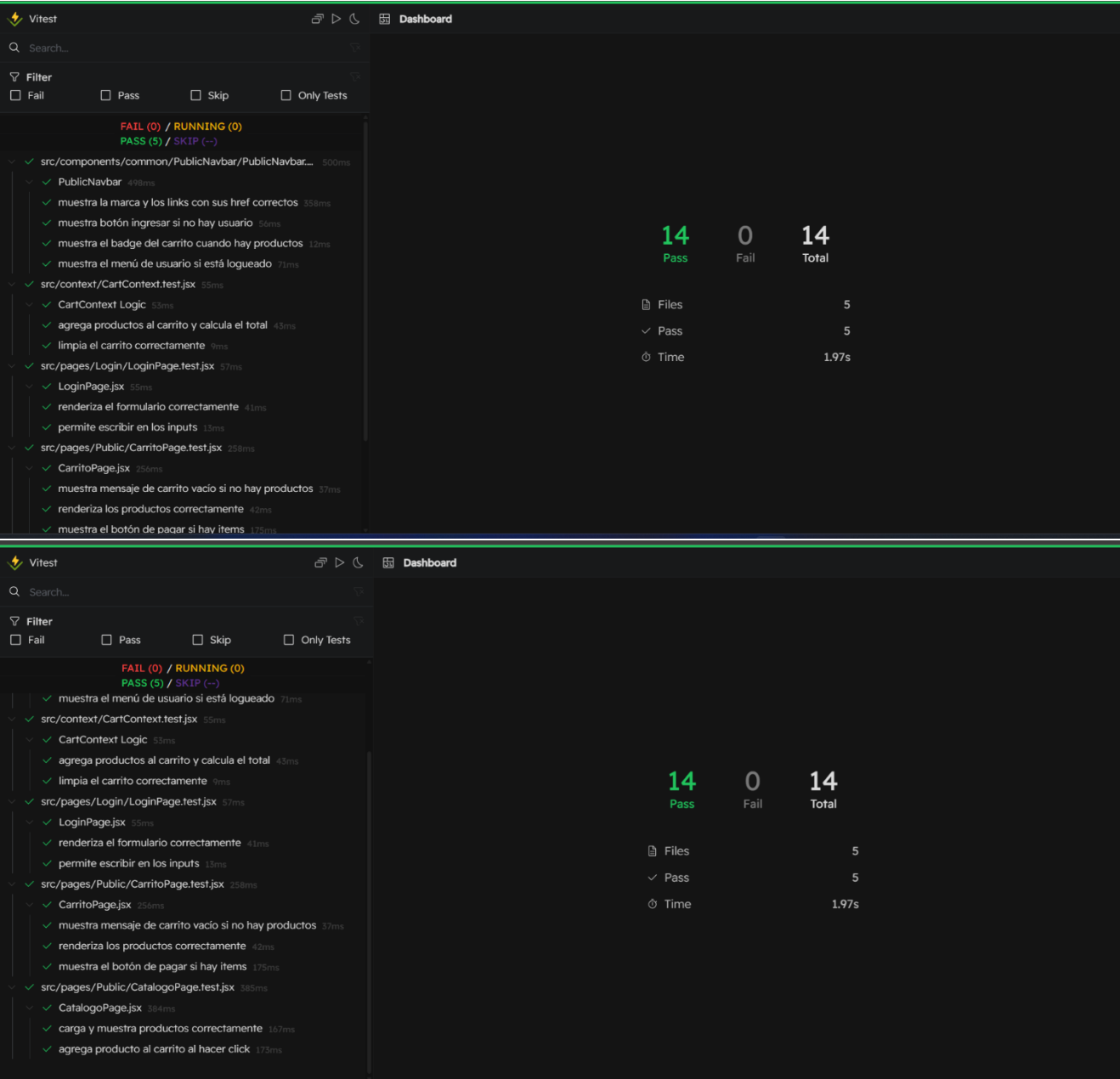
- **Archivo:** `src/context/CartContext.test.jsx`

- **Pruebas Realizadas:**

- Incremento del contador al agregar un ítem.
 - Reseteo del estado a 0 al limpiar el carrito.
 - Cálculo matemático correcto del total de la compra.
-

4. Resultados del Testing

La ejecución de la suite de pruebas fue exitosa, logrando un **100% de aprobación** en todos los escenarios planteados.



```

✓ src/context/CartContext.test.jsx (2 tests) 51ms
stderr | src/pages/Public/CarritoPage.test.jsx:52:5 > CarritoPage.jsx > muestra mensaje de carrito vacío si no hay productos
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7. You can use the `v7_startTransition` future flag to opt-in early. For more information, see https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the `v7_relativeSplatPath` future flag to opt-in early. For more information, see https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

stderr | src/pages/Login/LoginPage.test.jsx:30:5 > LoginPage.jsx > renderiza el formulario correctamente
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7. You can use the `v7_startTransition` future flag to opt-in early. For more information, see https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the `v7_relativeSplatPath` future flag to opt-in early. For more information, see https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

✓ src/pages/Login/LoginPage.test.jsx (2 tests) 84ms
stderr | src/pages/Public/CatalogoPage.test.jsx:38:3 > CatalogoPage.jsx > carga y muestra productos correctamente
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7. You can use the `v7_startTransition` future flag to opt-in early. For more information, see https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the `v7_relativeSplatPath` future flag to opt-in early. For more information, see https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the `v7_relativeSplatPath` future flag to opt-in early. For more information, see https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

✓ src/pages/Public/CarritoPage.test.jsx (3 tests) 281ms
stderr | src/components/common/PublicNavbar/PublicNavbar.test.jsx:29:5 > PublicNavbar > muestra la marca y los links con sus href correctos
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7. You can use the `v7_startTransition` future flag to opt-in early. For more information, see https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the `v7_relativeSplatPath` future flag to opt-in early. For more information, see https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

✓ src/components/common/PublicNavbar/PublicNavbar.test.jsx (4 tests) 469ms
  ✓ muestra la marca y los links con sus href correctos 337ms
✓ src/pages/Public/CatalogoPage.test.jsx (3 tests) 373ms

Test Files 5 passed (5)
Tests 14 passed (14)
Start at 20:32:53
Duration 4.03s (transform 716ms, setup 3.63s, collect 979ms, tests 1.26s, environment 11.29s, prepare 107ms)

PASS Waiting for file changes...
press h to show help, press q to quit

```

5. Conclusión

El proyecto FLASHU cuenta con una base sólida de pruebas automatizadas que cubren los flujos principales de usuario. La implementación de **Mocks** para la API y el Contexto ha permitido aislar componentes y probarlos de manera eficiente y confiable.

El sistema es **robusto** y escalable. Cualquier regresión o error introducido en futuras actualizaciones será detectado inmediatamente por esta suite de tests, garantizando la calidad del software a largo plazo.